
Регулярные выражения

Регулярные выражения

Регулярные выражения — это часть небольшой технологической области, невероятно широко используемой в огромном диапазоне программ. Регулярные выражения можно представить себе как мини-язык программирования, имеющий одно специфическое назначение: находить подстроки в больших строковых выражениях.

Регулярные выражения поддерживаются множеством классов .NET из пространства имен ***System.Text.RegularExpressions***.

Регулярные выражения

Язык регулярных выражений предназначен специально для обработки строк.

Язык включает два средства:

- Набор управляющих кодов для идентификации специфических типов символов
- Система для группирования частей подстрок и промежуточных результатов таких действий

Регулярные выражения

Главным преимуществом регулярных выражений является использование

метасимволов — специальные символы, задающие команды, а также управляющие последовательности, которые работают подобно управляющим последовательностям C#.

Это символы, предваренные знаком обратного слеша (\) и имеющие специальное назначение.

Классы символов

[...]	Любой из символов, указанных в скобках	[0-9] [0-9A-Fa-f]	<u>1</u> 2, <u>1</u> F, <u>4</u> B
[^...]	Любой из символов, не указанных в скобках	<[^>]>	<u><1></u> , <u><a></u> , <u><>></u>
.	Любой символ, кроме перевода строки или другого разделителя Unicode-строки		м.р "мир" или "мор"
\w	Любой алфавитно-цифровой символ, не являющийся пробелом, символом табуляции и т.п. Символы, образующие слово.	\w\w\w	<u>Г</u> од, <u>К</u> от, <u>К</u> урс
\W	Любой символ, не являющийся буквой и цифрой и не образующий слово	ИТ31\W	<u>ИТ31!</u> , <u>ИТ31?</u>
\s	Любой пробельный символ из набора Unicode		
\S	Любой непробельный символ из набора Unicode.		
\d	Любые ASCII-цифры. Эквивалентно [0-9]	ИТ\d\d	<u>ИТ31</u> , <u>ИТ311</u> , <u>ДИТ31</u>
\D	Любой символ, отличный от ASCII-цифр. Эквивалентно [^0-9]	926\D123	<u>926)123</u> , <u>1926-1234</u>

Символы повторения

{n,m}	Соответствует предшествующему шаблону, повторенному не менее n и не более m раз	$\backslash d\{2,4\}$	1, <u>12</u> , <u>123</u> , <u>1234</u> , 12345
{n,}	Соответствует предшествующему шаблону, повторенному n или более раз	$\backslash d\{3, \}$	1, 12, <u>123</u> , <u>1234</u> , <u>12345</u>
{n}	Соответствует в точности n экземплярам предшествующего шаблона	$\backslash d\{4\}$	1, 12, 123, <u>1234</u> , 12345
{,n}	Соответствует предшествующему шаблону, повторенному не более n раз	$\backslash d\{,2\}$	<u>1</u> , <u>12</u> , <u>123</u>
?	Соответствует нулю или одному экземпляру предшествующего шаблона; предшествующий шаблон является необязательным	кит?	<u>кит</u> , <u>киты</u> , <u>китов</u>
+	Соответствует одному или более экземплярам предшествующего шаблона	a\)+	<u>a</u>), <u>a</u>)), <u>a</u>))), <u>ba</u>))
*	Соответствует нулю или более экземплярам предшествующего шаблона	Эквивалентно {0,}	

По умолчанию квантификаторы жадные — захватывают максимально возможное число символов. Добавление ? делает их ленивыми, они захватывают минимально возможное число символов.

*?

+?

??

{m,n}?

{,n}?

{m,}?

```
string str =  
    @"(a + b) * (c + d) * (e + f)  
    (a + b) * (c + d) * (e + f)";
```

```
Regex pattern = new Regex(@"\(.*\)",
```

```
(a + b) * (c + d) * (e + f)  
(a + b) * (c + d) * (e + f)
```

```
Regex pattern =new Regex(@"\(.*?\) "
```

```
(a + b)  
(c + d)  
(e + f)  
(a + b)  
(c + d)  
(e + f)
```

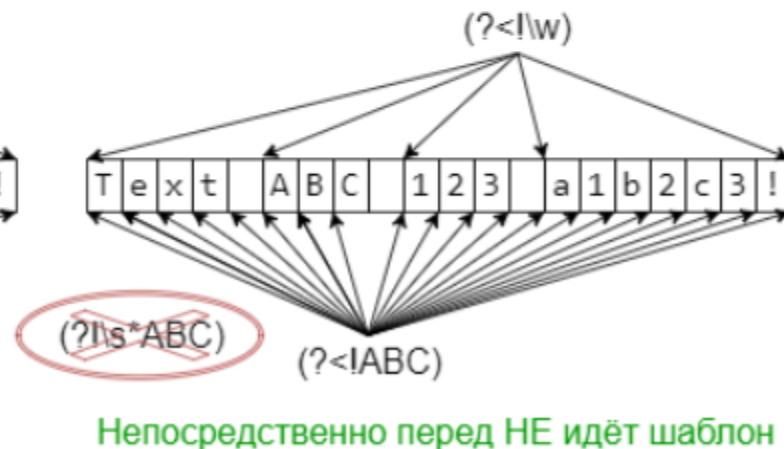
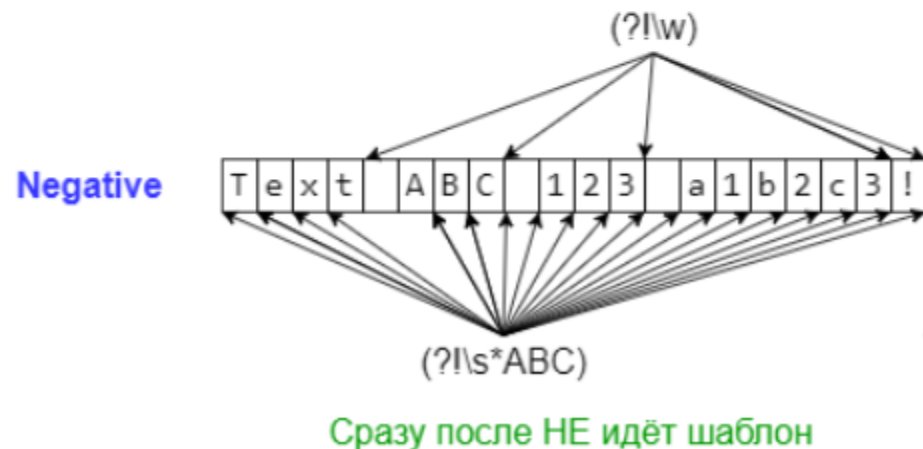
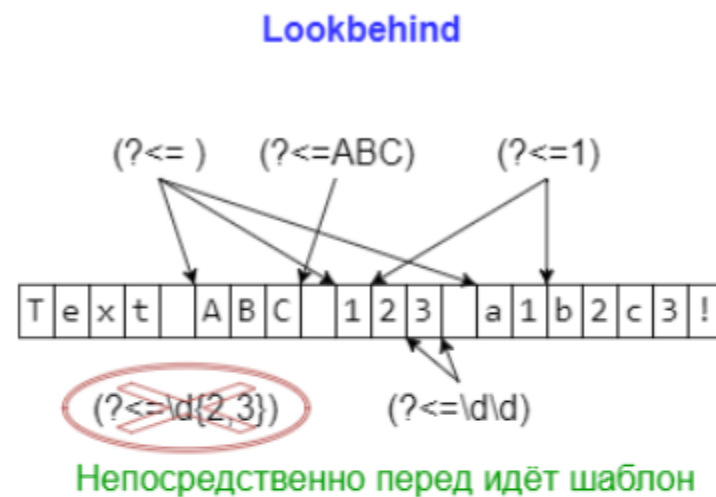
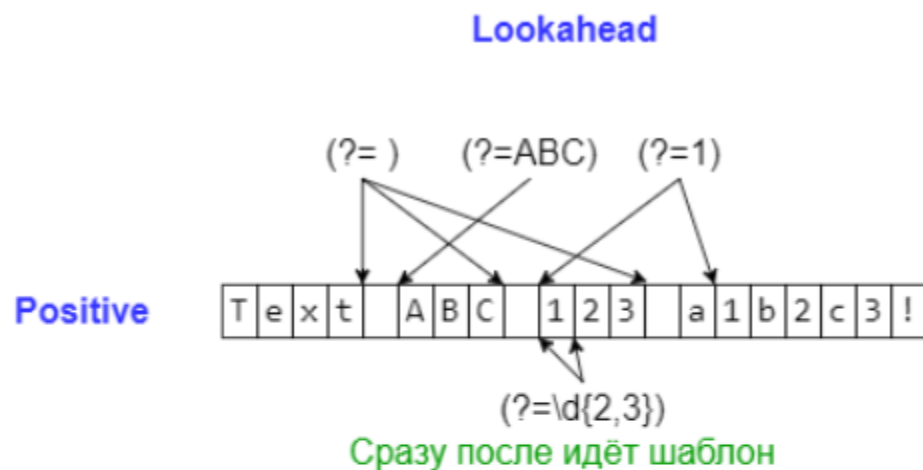
Символы регулярных выражений выбора

	Соответствует либо подвыражению слева, либо подвыражению справа (аналог логической операции ИЛИ).
(...)	Группировка. Группирует элементы в единое целое, которое может использоваться с символами *, +, ?, и т.п. Также запоминает символы, соответствующие этой группе для использования в последующих ссылках.
(?:...)	Только группировка. Группирует элементы в единое целое, но не запоминает символы, соответствующие этой группе.
(?<name>...)	Группировка с указанием имени группы

Сложные шаблоны, соответствующие позиции

Шаблон	Описание	Пример	Применяем к тексту
(?=...)	<i>lookahead assertion</i> , соответствует каждой позиции, сразу после которой начинается соответствие шаблону ...	ИТ (?=31)	<u>ИТ</u> 31, ИТ 41
(?!...)	<i>negative lookahead assertion</i> , соответствует каждой позиции, сразу после которой НЕ может начинаться шаблон ...	ИТ (?!31)	ИТ 31, <u>ИТ</u> 41
(?<=...)	<i>positive lookbehind assertion</i> , соответствует каждой позиции, которой может заканчиваться шаблон ... Длина шаблона должна быть фиксированной, то есть abc и a b — это ОК, а a* и a{2,3} — нет.	(?<=abc)def	abc <u>def</u> , bcdef
(?<!...)	<i>negative lookbehind assertion</i> , соответствует каждой позиции, которой НЕ может заканчиваться шаблон ...	(?<!abc)def	abcdef, bc <u>def</u>

Сложные шаблоны, соответствующие позиции



Якорные символы регулярных выражений

^	Соответствует началу строкового выражения или началу строки при многострочном поиске.	@'^пр\w*"	"привет" в строке "привет мир"
		^Hello	"Hello, IT-31", но не "Ok, Hello IT-31" т.к. в этой строке слово "Hello" находится не в начале
\$	Соответствует концу строкового выражения или концу строки при многострочном поиске.	@'\w*ир\$"	"мир" в строке "привет мир"
		Hello\$	"IT-31, Hello"
\b	Начало или конец слова (слева пусто или не-буква, справа буква и наоборот). В отличие от предыдущих соответствует позиции, а не символу	a\b	aaa a aaa
		\ba	aaa aaa
		\b(my)\b	В строке "Hello my world" выберет слово "my"
\B	Не граница слова: либо и слева, и справа буквы, либо и слева, и справа НЕ буквы	\Ba\B	aaa a aaa
		\B(ld)\B	Соответствие найдется в слове "World", но не в слове "ld"

Спецсимволы экранирования в регулярных выражениях

Выражение	Соответствие
\	не соответствует ничему, только экранирует следующий за ним символ. Это нужно, если вы хотите ввести метасимволы !\$()*+.<>?[\]^{} в качестве их буквальных значений.
\Q	не соответствует ничему, только экранирует все символы вплоть до \E
\E	не соответствует ничему, только прекращает экранирование, начатое \Q

Примеры

```
string s = "Бык тупогуб, тупогубенький бычок, у быка губа белая была тупа";
```

```
Regex regex = new Regex(@"\w*губ\w*");
```

Так как выражение `\w*` соответствует любой последовательности алфавитно-цифровых символов любой длины, то данное выражение найдет все слова, содержащие корень "губ".

Нахождение телефонного номера в формате 111-111-1111:

```
string s = "456-435-2318";  
Regex regex = new Regex(@"\d{3}-\d{3}-\d{4}");
```

```
string s = "456-435-2318";  
Regex regex = new Regex("[0-9]{3}-[0-9]{3}-[0-9]{4}");
```

Примеры

Можно задать диапазон для алфавитных символов:

Regex regex = new Regex("[a-v]{5}") - данное выражение будет соответствовать любому сочетанию пяти символов, в котором все символы находятся в диапазоне от a до v.

Regex regex = new Regex(@"[2]*-[0-9]{3}-\d{4}") - выражение будет соответствовать, например, такому номеру телефона "222-222-2222" (так как первые числа двойки)

С помощью операции | можно задать альтернативные символы:

Regex regex = new Regex(@"[2|3]{3}-[0-9]{3}-\d{4}") - первые три цифры могут содержать только двойки или тройки. Такой шаблон будет соответствовать, например, строкам "222-222-2222" и "323-435-2318". А вот строка "235-435-2318" уже не подпадает под шаблон, так как одной из трех первых цифр является цифра 5.

Если надо найти, строки, где содержится точка, звездочка или какой-то другой специальный символ, надо экранировать эти символы слешем:

```
Regex regex = new Regex(@"[2|3]{3}\.[0-9]{3}\.\d{4}");  
// этому выражению будет соответствовать строка "222.222.2222"
```

Регулярные выражения

IsMatch() - проверяет, существует ли текст, соответствующий шаблону, в исходной строке

Match() - возвращает объект класса *Match*, содержащий сведения о первой подстроке, которая сопоставлена шаблону регулярного выражения

Matches() - возвращает объект класса **MatchCollection**, который, в свою очередь, содержит сведения обо всех совпадениях, которые обработчик регулярных выражений находит во входной строке.

Свойство **Success** возвращает значение *true*, если найдено следующее совпадение, которое можно получить с помощью вызова метода **Match.NextMatch()**.

Эти вызовы метода можно продолжать пока свойство **Match.Success** не вернет значение *false*.

Информации о перечислении *RegexOptions*, экземпляр которого можно передать конструктору класса *Regex*

Член	Описание
CultureInvariant	Предписывает игнорировать национальные установки строки
ExplicitCapture	Модифицирует способ поиска соответствия, обеспечивая только буквальное соответствие
IgnoreCase	Игнорирует регистр символов во входной строке
IgnorePatternWhitespace	Удаляет из строки не защищенные управляющими символами пробелы и разрешает комментарии, начинающиеся со знака фунта или хеша
Multiline	Изменяет значение символов ^ и \$ так, что они применяются к началу и концу каждой строки, а не только к началу и концу всего входного текста
RightToLeft	Предписывает читать входную строку справа налево вместо направления по умолчанию — слева направо (что удобно для некоторых азиатских и других языков, которые читаются в таком направлении)
Singleline	Специфицирует однострочный режим, в котором точка (.) символизирует соответствие любому символу


```
string[] test = {
    "Группа", "Группа ИТ31", "ИТ-41 группа"};

// Проверим, содержится ли в исходных строках слово Группа
Regex regex = new Regex("Группа");

    Console.WriteLine("Регистрозависимый поиск: ");
    foreach (string str in test)
    {
if (regex.IsMatch(str))
    Console.WriteLine("В исходной строке: \"{0}\" есть совпадения!", str);
    }

// Теперь укажем поиск, не зависимый от регистра
regex = new Regex("Группа", RegexOptions.IgnoreCase);
Console.WriteLine("Регистронезависимый поиск: ");
    foreach (string str in test)
    {
if (regex.IsMatch(str))
    Console.WriteLine("В исходной строке: \"{0}\" есть совпадения!", str);
    }
```

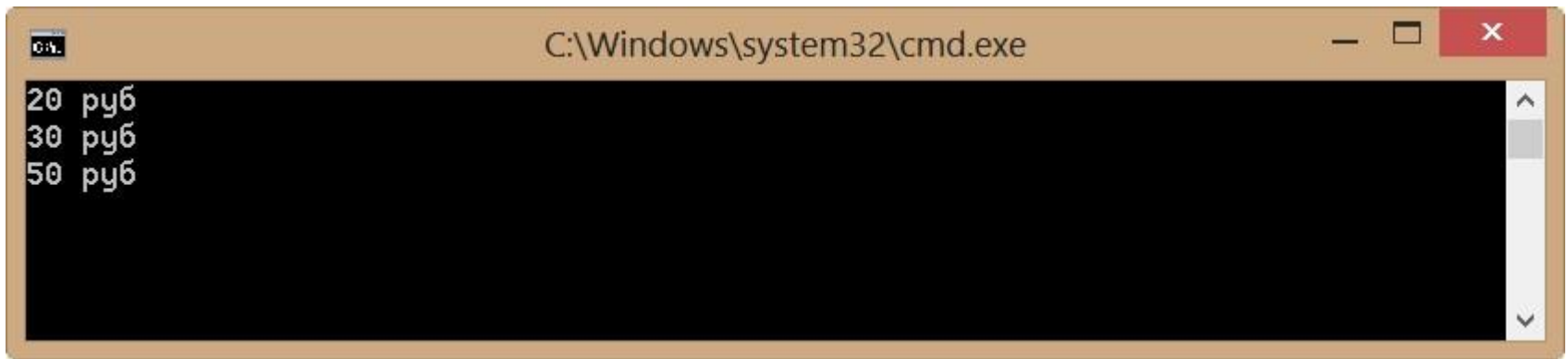
```
string input = "Наши цены: " +  
    "1 кг. яблок - 200 руб. " +  
    "2 кг. апельсинов - 300 руб. " +  
    "0.5 кг. орехов - 500 руб.";  
  
string pattern = @"\b(\d+\s?руб)";  
Regex regex = new Regex(pattern);  
  
// Получаем совпадения в экземпляре класса Match  
Match match = regex.Match(input);  
  
// отображаем все совпадения  
while (match.Success)  
{  
    // Т.к. мы выделили в шаблоне одну группу (одни круглые скобки),  
    // ссылаемся на найденное значение через свойство Groups класса Match  
    Console.WriteLine(match.Groups[1].Value);  
    // Переходим к следующему совпадению  
    match = match.NextMatch();  
}
```

```
string input = "Добро пожаловать в наш магазин, вот наши цены: " +  
    "1 кг. яблок - 20 руб. " +  
    "2 кг. апельсинов - 30 руб. " +  
    "0.5 кг. орехов - 50 руб.";
```

```
string pattern = @"\b(\d+\W?руб)";  
Regex regex = new Regex(pattern);
```

// Достигаем того же результата что и в предыдущем примере,
используя метод `Regex.Matches()` возвращающий `MatchCollection`

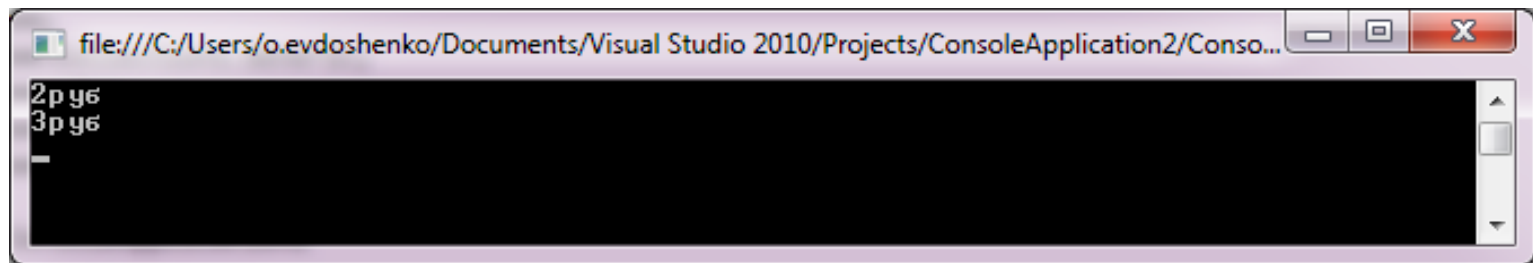
```
foreach (Match match in regex.Matches(input))  
{  
    Console.WriteLine(match.Groups[1].Value);  
}
```



```
C:\Windows\system32\cmd.exe
20 руб
30 руб
50 руб
```

```
string input = "Добро пожаловать в наш магазин, вот наши цены: " +  
               "1 кг. яблок - 2руб. " +  
               "2 кг. апельсинов - 3руб. " +  
               "0.5 кг. орехов - 50руб.";
```

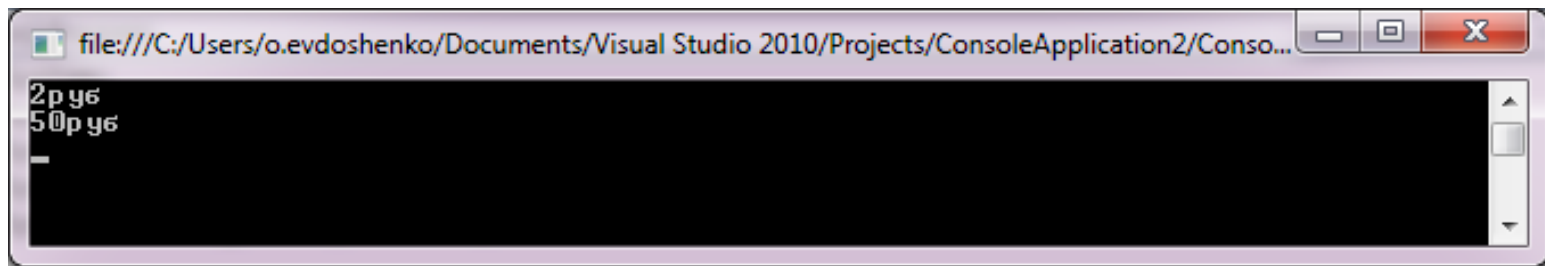
```
string pattern = @"\b(\d?\W?руб)";
```



```
file:///C:/Users/o.evdoshenko/Documents/Visual Studio 2010/Projects/ConsoleApplication2/Conso...
2p yб
3p yб
-
```

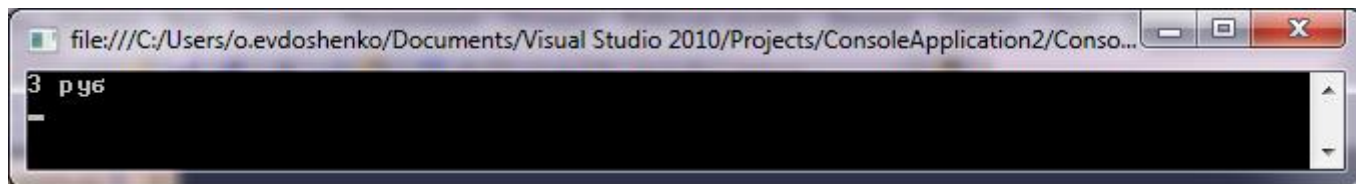
```
string input = "Добро пожаловать в наш магазин, вот наши цены: " +  
    "1 кг. яблок - 2руб. " +  
    "2 кг. апельсинов - 3 руб. " +  
    "0.5 кг. орехов - 50руб.";
```

```
string pattern = @"\b(\d+руб)";
```



```
string input = "Добро пожаловать в наш магазин, вот наши цены: " +  
    "1 кг. яблок - 2руб. " +  
    "2 кг. апельсинов - 3 руб. " +  
    "0.5 кг. орехов - 50руб.";
```

```
string pattern = @"\b(\d+\W+руб)";
```



```
string str =  
    @"Обычная (случайная) строка  
      (последовательность символов).  
      И еще просто набор цифр - (как число)";  
  
Regex pattern =  
    new Regex(@"\((?<val>.*?)\)",  
        RegexOptions.IgnoreCase |  
        RegexOptions.Singleline);  
  
foreach (Match m in pattern.Matches(str))  
  
    if (m.Success)  
        Console.WriteLine(m.Groups["val"].Value);
```

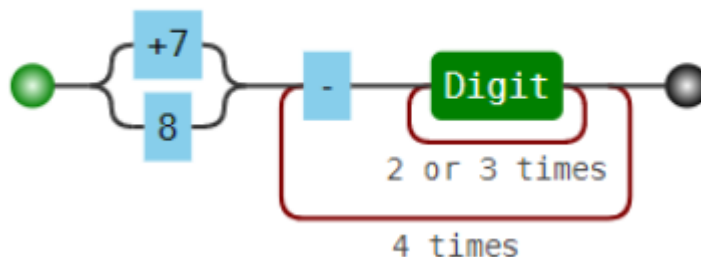
`\b\w{3}\b` Слова в точности из трёх букв
`\b` означает границу слова
(с одной стороны буква, а с другой — нет)
`\w` — любая буква,
`{3}` — ровно три раза

`[-+]? \d+` Целое число, например, 7, +17, -42, 0013
(возможны ведущие нули)
`[-+]?` — либо -, либо +, либо пусто
`\d+` — последовательность из 1 или более цифр

`(?:\+7|8)(?:-\d{2,3}){4}`

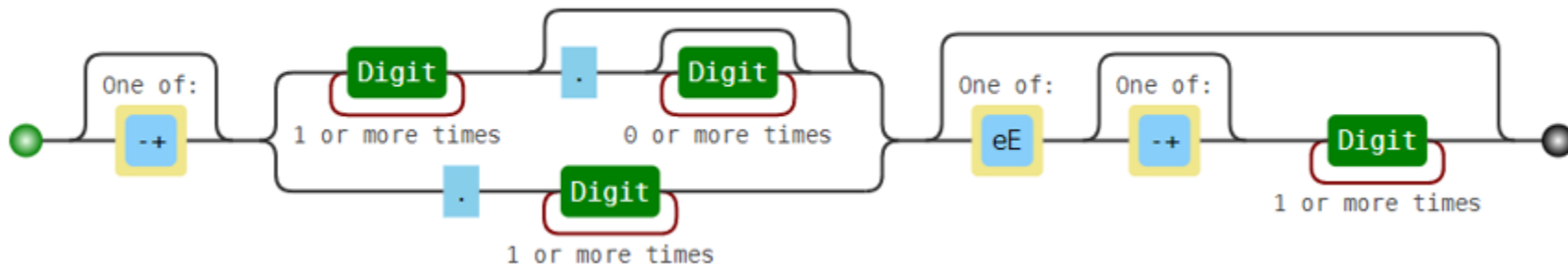
+7-926-123-12-12, 8-926-123-12-12

RegExp: `/(\?:\+7|8)(\?:-\d{2,3}){4}/`

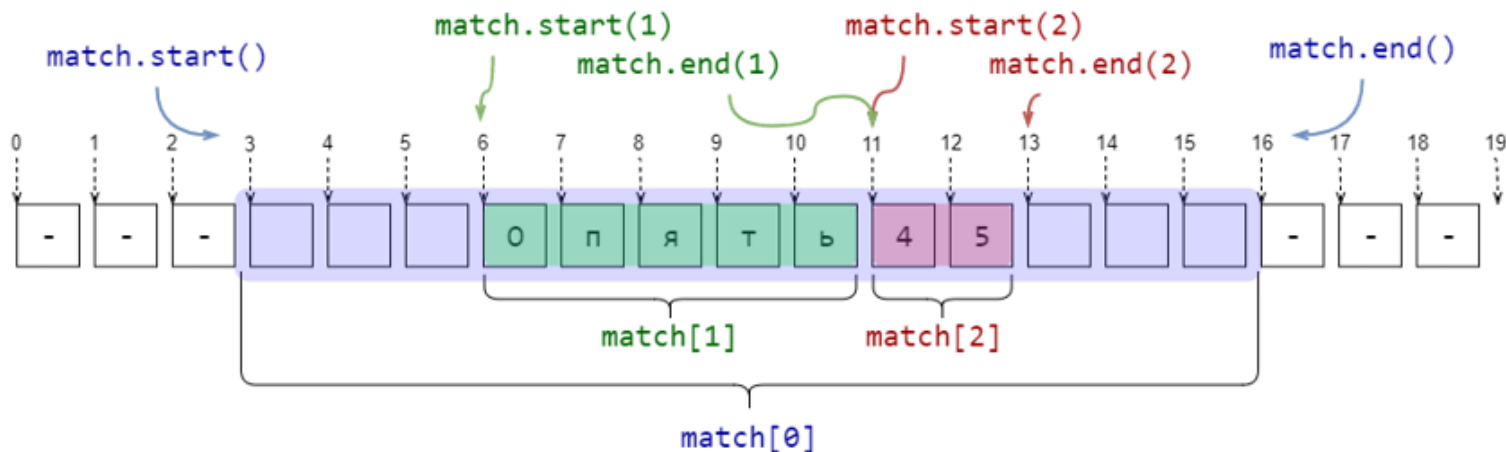
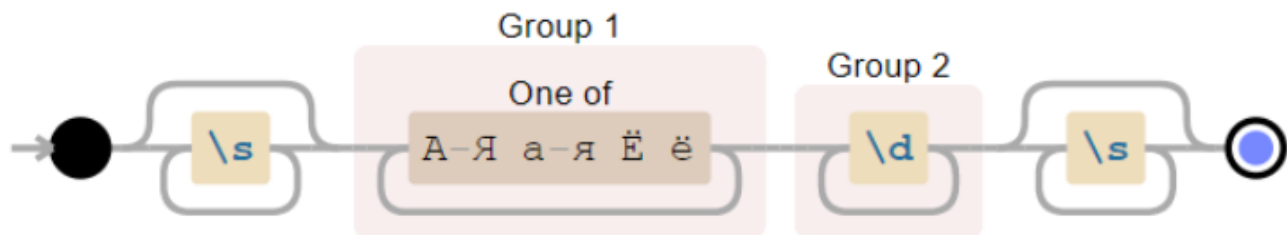


Действительное число, возможно в экспоненциальной записи
 Например, 0.2, +5.45, -.4, 6e23, -3.17E-14.

RegExp: `/[-+]?(?:\d+(?:\.\d*)?|\.\d+)(?:[eE][-+]?\d+)?/`



`\s*([А-Яа-яЁё]+\d+)\s*`



Regex.Replace() - извлекает совпадения в исходной строке, и заменяет их на собственные значения.

Символ	Описание	Пример шаблона	Пример шаблона замены	Результат (входная -> результирующая строки)
\$number	Замещает часть строки, соответствующую группе number	\b(\w+)(\s)(\w+)\b	\$3\$2\$1	"один два" -> "два один"
\$` (апостроф)	Замещает весь текст входной строки до соответствия	B+	\$`	"AABBCC" -> "AAAACC"
\$'	Замещает весь текст входной строки после соответствия	B+	\$'	"AABBCC" -> "AACCCC"

```
output = Regex.Replace(input, "B+", "$'");
```

```
string input = "Добро пожаловать в наш магазин, вот наши цены: \n" +  
    "\t 1 кг. яблок - 200 руб. \n" +  
    "\t 2 кг. апельсинов - 300 руб. \n" +  
    "\t 0.5 кг. орехов - 500 руб. \n";
```

```
Console.WriteLine("Исходная строка:\n {0}", input);
```

```
// В шаблоне используются 2 группы
```

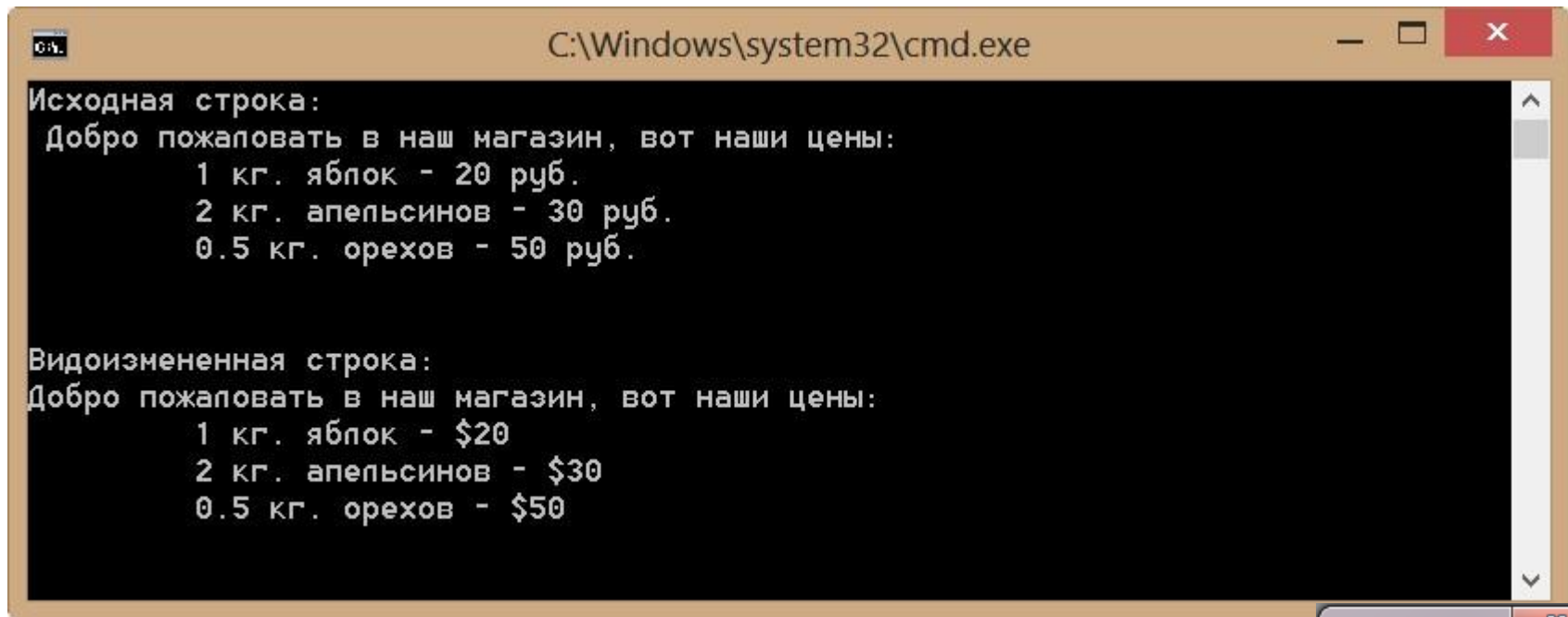
```
string pattern = @"\\b(\\d+)\\W?(руб.)";
```

```
// Строка замены "руб." на "$"
```

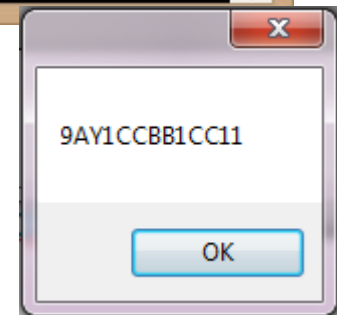
```
string replacement1 = "$$$1";
```

```
// Перед первой группой ставится знак $, вторая группа удаляется без  
замены
```

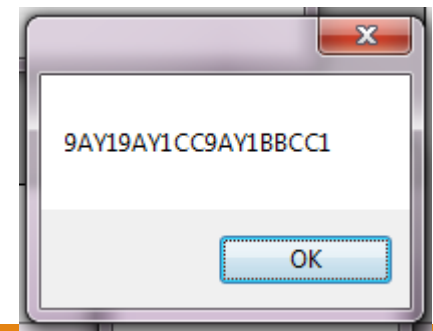
```
input = Regex.Replace(input, pattern, replacement1);
```



```
string input = "9AY1BBCCBB1";  
input = Regex.Replace(input, "B+", "$'");  
MessageBox.Show(input);
```



```
string input = "9AY1BBCCBB1";  
input = Regex.Replace(input, "B+", "$`");  
MessageBox.Show(input);
```



Задания:

Написать регулярное выражение для выбора веса продуктов:

- 1) Вес может быть как целым числом, так и с плавающей точкой**
- 2) В качестве разделителя между целой и дробной частью может быть точка или запятая**
- 3) После веса пробел и далее указания размерности – кг.**

```
string pattern = @"\\b(\\d+[,.]?\\d*\\W?кг.)";
```